NUS Orbital 2026: NUSell

Final README

Team: Karang Guni

Proposed Level of Achievement: Project Apollo

Contents

1. Motivation
2. Aim
3. User Stories
4. Features
5. System Design & Architecture
6. Tech Stack
7. Software Engineering Practices
8. System and Integrated testing
9. User Testing
10. Automated Testing
11. Development Plan
12. Proof of Concept
13. Link to our Application

1. Motivation

Every year, students move in and out of campus housing, often leaving behind usable furniture, electronics, and clothing that ultimately end up discarded. This contributes to unnecessary waste and missed opportunities for other students who could still use these items and benefit from them at a lower cost.

Currently, NUS students rely on informal channels such as Telegram groups (e.g. Eusoff Bazaar, NUS Marketplace) to buy and sell second-hand items. While these channels share a similar goal to NUSell, they have several limitations:

- Listings are unstructured and mixed with unrelated messages, making it difficult for users to browse
- There is no search or filter functionality. Buyers must scroll through long chat histories to find relevant items
- No accountability mechanism. Buyers have no way to verify seller reliability
- Listings have limited reach beyond the chats they are posted in

Drawing inspiration from platforms like Carousell, we aim to build NUSell, a dedicated campus-specific web application that connects NUS buyers and sellers.

2. Aim

By creating a convenient, campus-specific platform, we aim to:

- Make it easier for NUS students to give their items a second life
- Connect buyers directly with sellers within NUS
- Cultivate a circular economy within NUS to promote sustainability
- Provide a structured alternative to existing solutions

How NUSell Differs From Existing Solutions:

NUS Marketplace (Telegram)

- Unstructured chat format: cluttered, no filtering, listings get lost among other messages, results in little and effective reach. Uncertainty regarding the trustworthiness of sellers
- NUSell provides an effective, one-stop platform where items can be easily filtered according to user needs

Hostel channels eg. Eusoff Bazaar (Telegram)

- Limited to that particular hostel, and shares the same structural limitations as above.
- NUSell creates a centralised platform for all students regardless of hostel or faculty.

Carousell

- Not campus-specific — sellers and buyers are not verified NUS students, and there are no campus-focused features such as meetup locations within NUS.
- NUSell targets students in NUS, features a much more curated assortment of items, and facilitates easy transactions within campus.

3. User Stories

- As a student who wants to sell their second-hand items, I want to be able to put up a listing on the system, which will connect me to buyers so I can complete the transaction.
- As a student looking to purchase something specific, I want to be able to search and filter the listing gallery (by item type, price range, meeting location) so I can find the best match for what I am looking for.
- As a seller, I want to have a profile page that shows all my current listings and my accumulated reputation points so buyers can see my track record.
- As a buyer who has completed a transaction, I want to be able to award the seller points based on the transaction experience so that trustworthy sellers are rewarded with higher visibility.

4. Features

To achieve the above, we are targeting a Reputation System and a Giveaway section on top of the core marketplace loop of creating listings, browsing, filtering by price or category etc.

Feature 1: Authentication (Login & Register)

Every buyer and seller needs their own account before they can list or contact anyone, so this is the entry point into NUSell and the first feature we built.

We implemented a Login page and a separate Register page so users can create and access accounts using a username, NUS email, Telegram handle, and password. Passwords are hashed with bcrypt before being stored, so the database never holds a raw password. On a successful login or registration, the backend signs a JSON Web Token (JWT), which the frontend stores in localStorage and uses to stay authenticated across page reloads. AuthContext holds the logged-in user, and a PrivateRoute wrapper redirects unauthenticated visitors back to the Login page.

Implementation Philosophy:

- We kept Login and Register as distinct pages so account-creation logic is isolated from authentication.
- We chose JWTs over server-side sessions so the backend stays stateless and can verify any request from the token alone.

Challenges:

- Keeping the user logged in across refreshes: solved by re-validating the stored token against a GET /api/auth/me request when the app loads.
- Giving clear error feedback for duplicate emails/usernames and for incorrect credentials.

Current progress: Complete

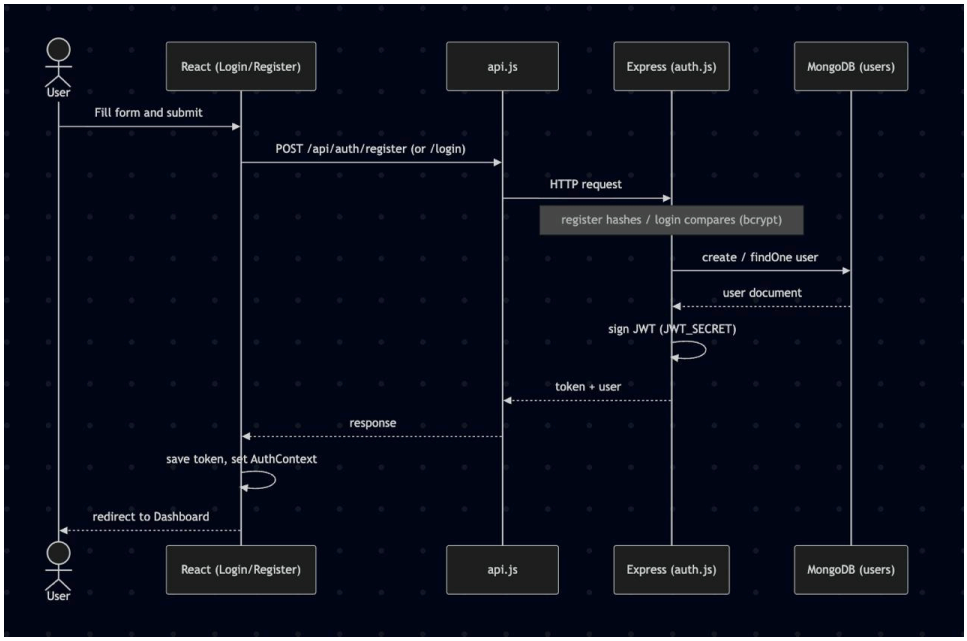


Figure 1: Authentication (login/register) sequence diagram.

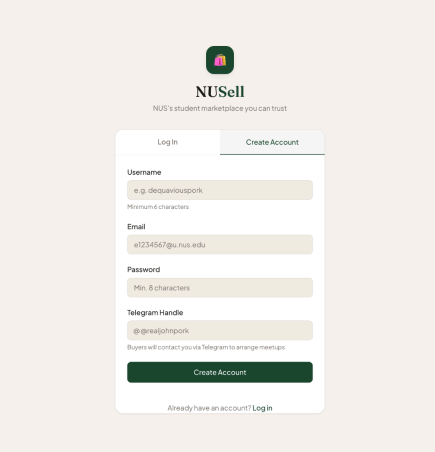


Figure 2a: Account creation screen.

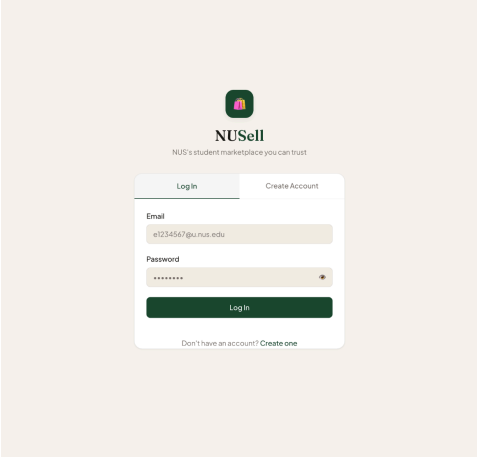


Figure 2b: Account login screen.

Feature 2: Item Listings (Create, Edit & Delete)

Listings are the core unit of NUSell. Sellers enter the item name, price, condition, category, meetup location, description, and an optional image through a Create Listing form. The listing is tied to the logged-in user as its seller (read from the verified JWT, not from the form) so a user cannot create a listing on someone else's behalf. On submission, the new listing is saved to the database and the seller is redirected to the listing's detail page.

On a listing's detail page, the owner sees Edit and Delete controls, which are hidden from everyone else. Editing reuses the create form, pre-filled with the listing's current data, so the same component powers both flows. Deleting is a soft delete (isActive set to false): the record is preserved and simply removed from the feed rather than permanently destroyed, which keeps data intact for any future transaction history. The backend re-checks ownership on every edit and delete, returning a 403 Forbidden response if the requester is not the seller.

Implementation Philosophy:

- The seller is derived from the token rather than trusted from the client, which is both simpler and more secure.

- The same form component powers both creating and editing, reducing duplication (DRY principle).
- Ownership is enforced on the server, not merely hidden on the client, so the rule cannot be bypassed by calling the API directly.

Challenges:

- Validating the form and giving clear feedback for missing or invalid fields (e.g. a negative price or an empty title).
- Ensuring only the owner can modify a listing even if a request is sent directly to the API outside the UI.

Current progress: Complete

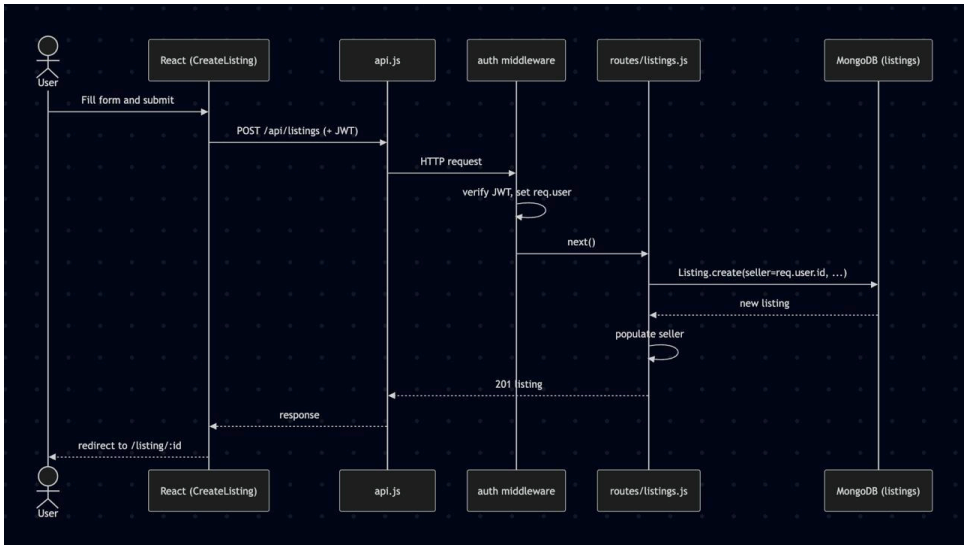


Figure 3a: Create Listing sequence diagram.

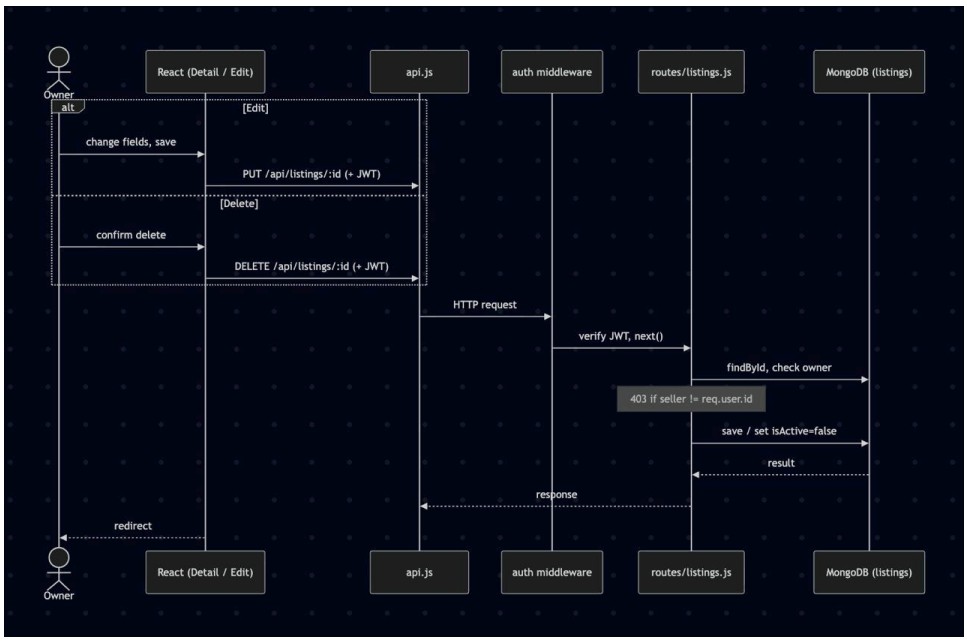


Figure 3b: Edit / Delete Listing sequence diagram (owner only).

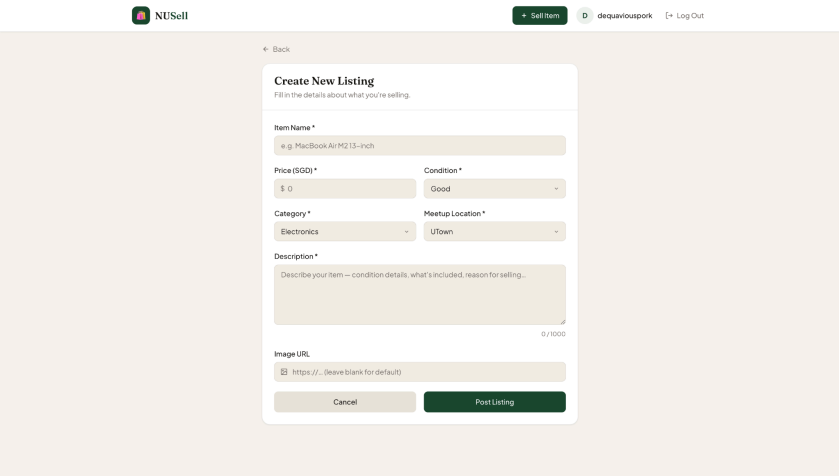


Figure 4a: Create Listing form.

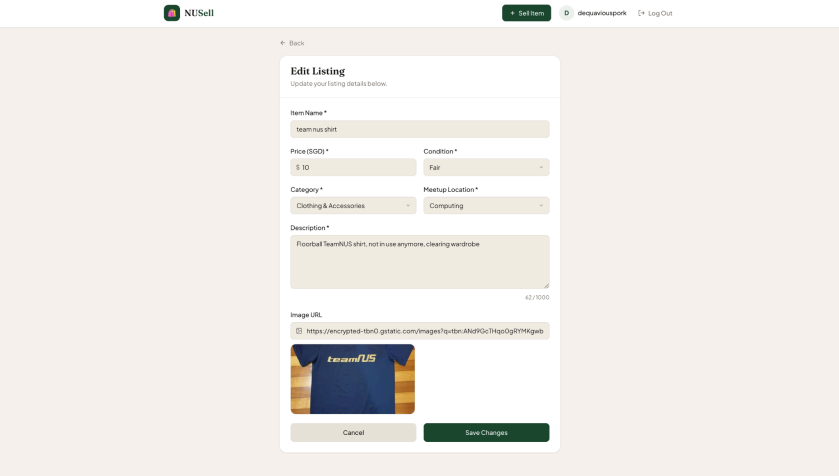


Figure 4b: Edit Listing form, pre-filled with existing data.

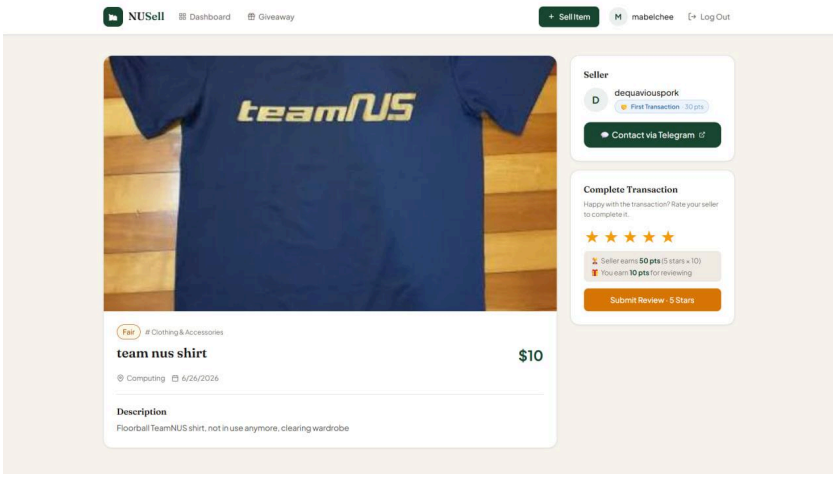


Figure 4c: Listing detail page with owner Edit / Delete controls.

Feature 3: Gallery, Search & Filter

The Dashboard displays all active listings in a responsive gallery of cards showing the item's image, title, price, condition, and location. Buyers can search by item name and apply filters for category, price range, condition, and meetup location, and clicking a card opens that listing's detail page.

Implementation Philosophy:

- Only active (non-deleted) listings are returned, sorted newest-first, so the feed always shows current items.
- Filtering runs on the client for instant feedback on the current result set, while the backend also supports query-parameter filtering so the same endpoint can scale to larger datasets.

Challenges:

- Combining several filters cleanly and keeping the results responsive as the user types.

Current progress: Complete

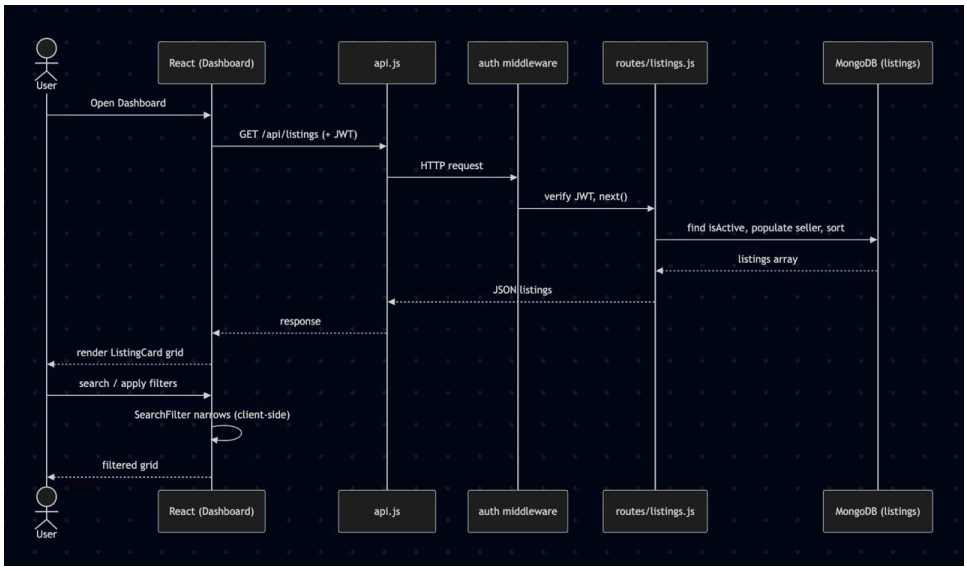


Figure 5: Browse and filter sequence diagram.

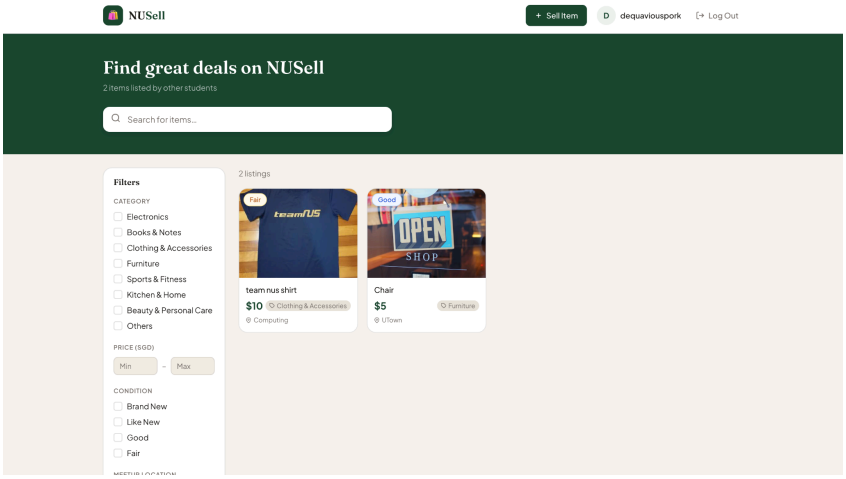


Figure 6a: Dashboard showing all active listings.

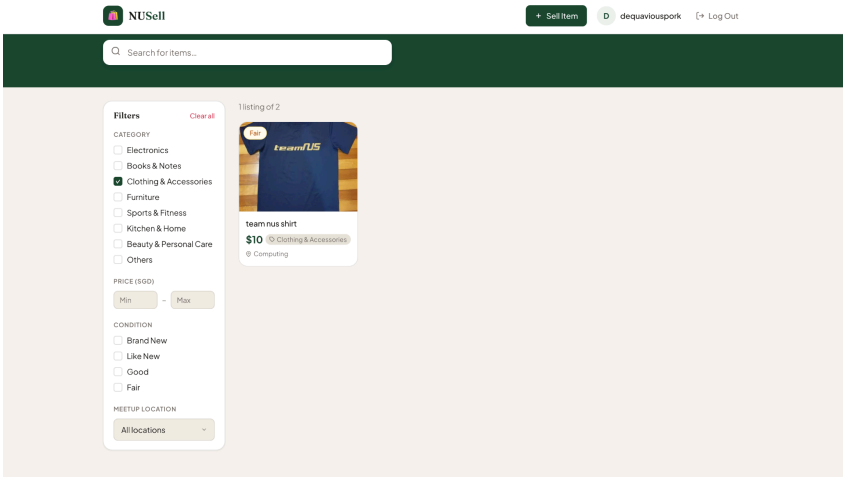


Figure 6b: Gallery narrowed by a category filter

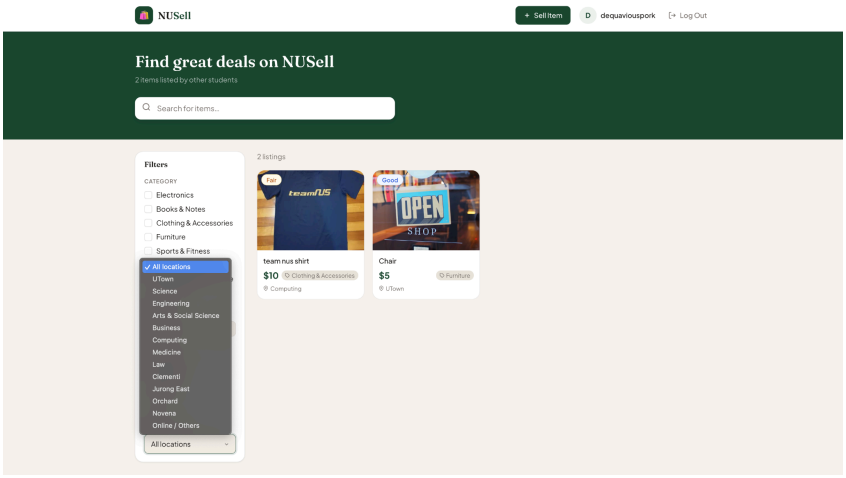


Figure 6c: Meetup-location filter options

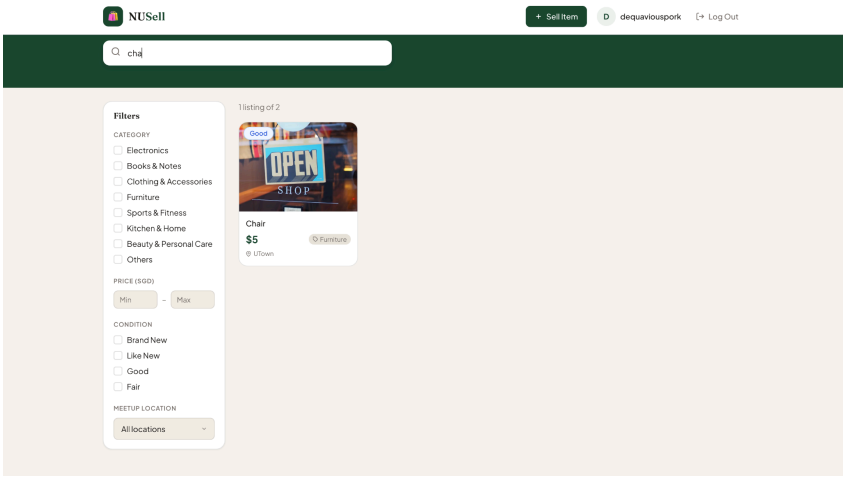


Figure 6d: Gallery narrowed by a search term

Feature 4: Profile Pages

Each user has a profile page showing their username, Telegram handle, and their active listings. The page fetches the user's public information and their listings in parallel (Promise.all) for a single, fast render. Owners can edit their own profile (username and Telegram handle) through an in-page modal, and saved changes update both the profile and the navbar immediately. Public profile responses deliberately exclude sensitive fields such as email and password.

Implementation Philosophy:

- Profile data and listings are fetched together so the page renders in one pass rather than in stages.
- Editing happens in a modal rather than on a separate page, keeping the user in context.

Challenges:

- Keeping the navbar's username in sync after an edit: solved with a shared updateUser helper in AuthContext.

Current progress: Complete.

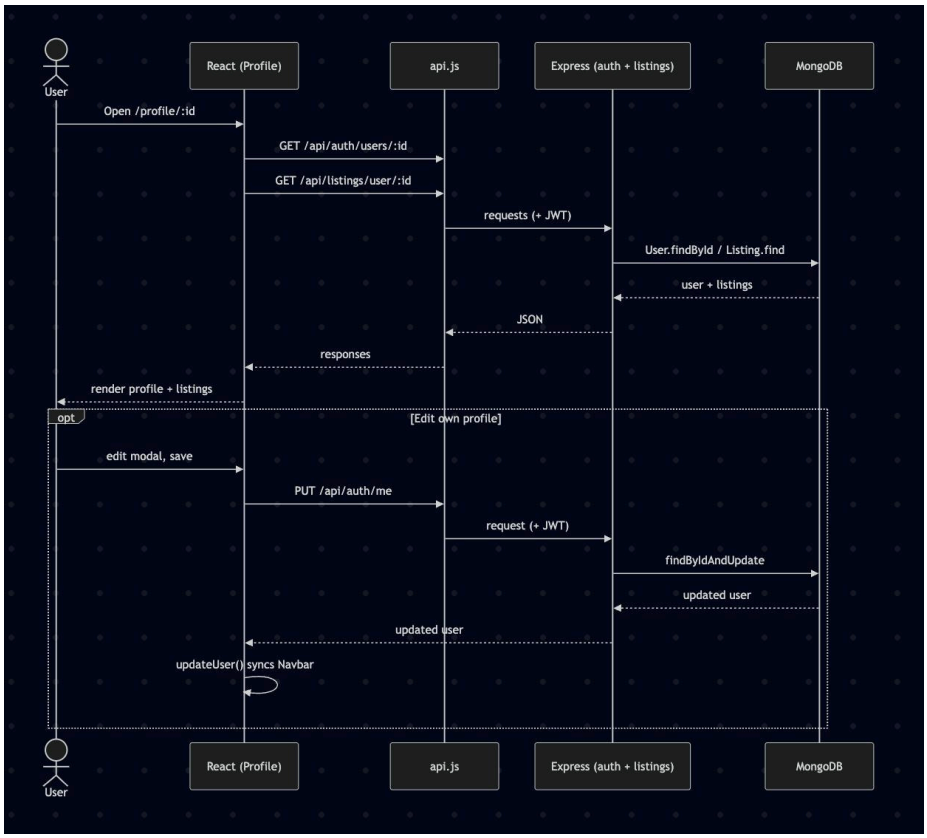


Figure 7: View and edit profile sequence diagram

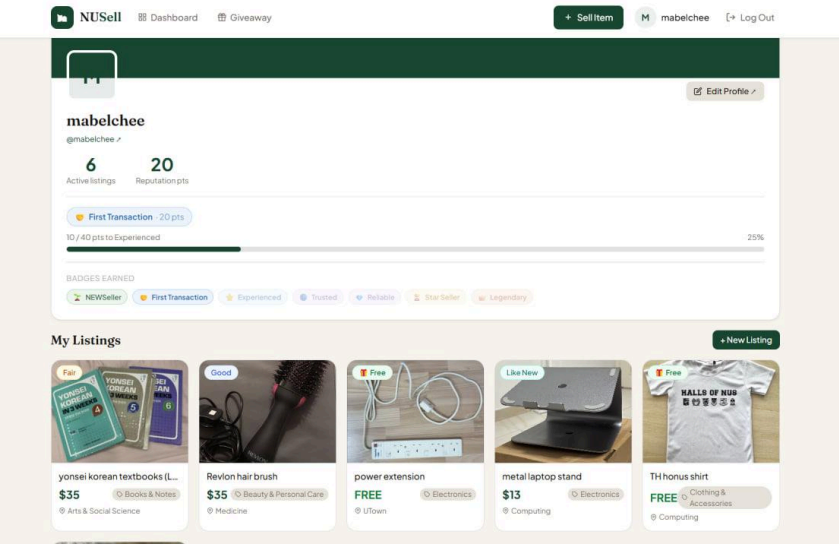


Figure 8a: Own profile page

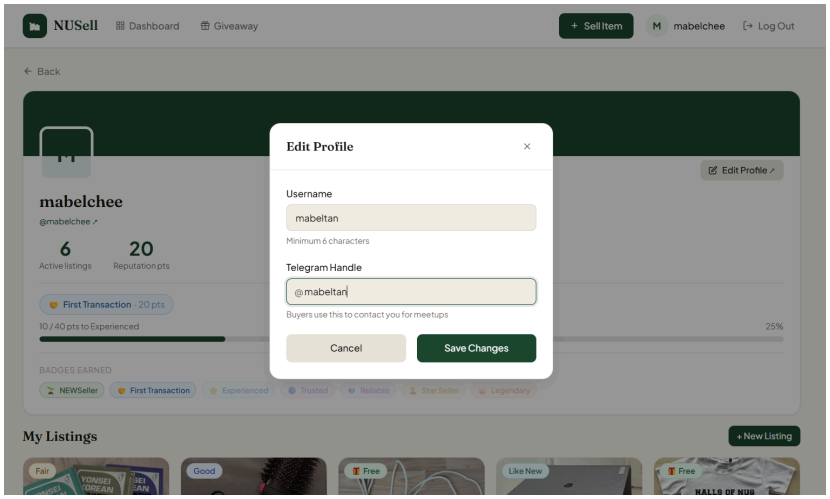


Figure 8b: Editing profile details

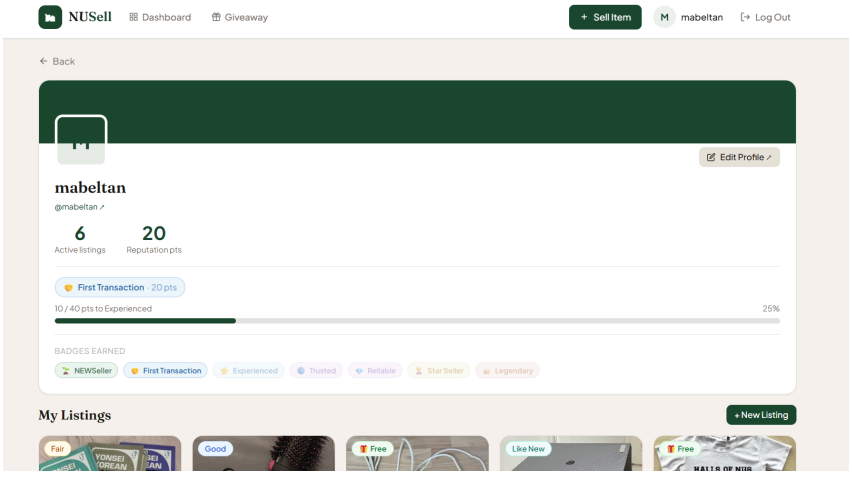


Figure 8c: Profile after a saved edit

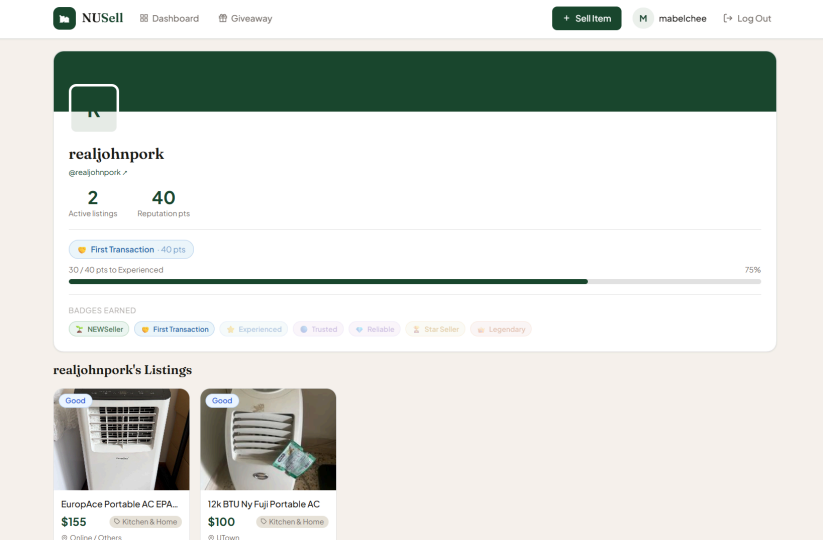


Figure 8d: Another user's public profile page.

Feature 5: Giveaway Section

A separate page from the main gallery where sellers can list their items for free. These listings will be gathered on a dedicated Giveaway page, allowing buyers to easily filter and browse free items available for collection.

Current progress: Completed

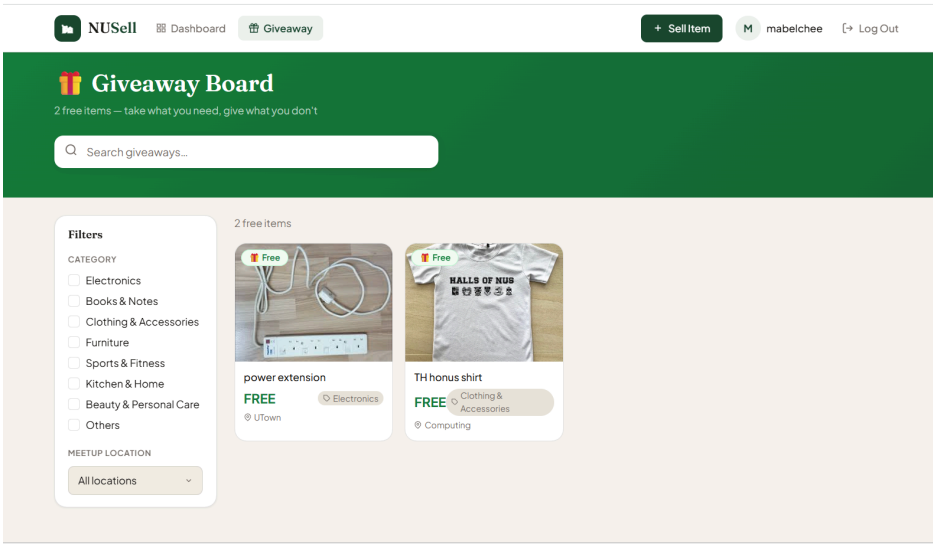


Figure 9: Giveaway section

Feature 6: Reputation System

After each successful transaction, buyers will be able to award the seller points (1–5) based on satisfaction with the transaction. Points will accumulate on the seller's profile, unlock levels and badges, and boost listing visibility in filtered search results.

Current progress: Complete

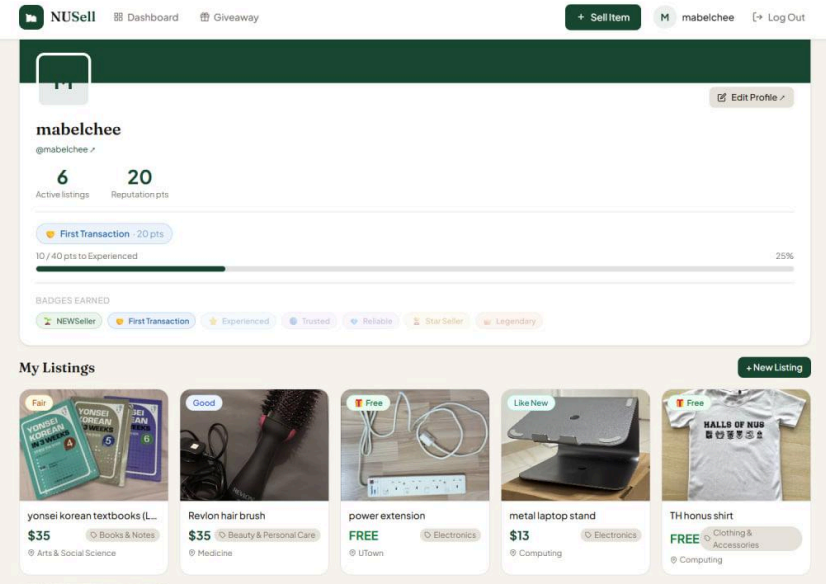


Figure 10a: Reputation badges and progress bar on profile

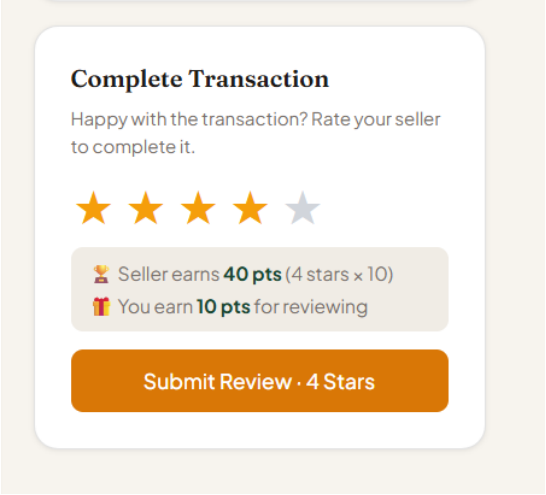


Figure 10b: Review component to award and earn points.

5. System Design & Architecture

NUSell follows a standard three-tier web application architecture:

- Frontend (React): Handles all user interface rendering and client-side routing. Components include the Home/Feed page, Listing Detail page, Create/Edit Listing form, User Profile page, and the Giveaway page.

- Backend (Node.js + Express): Provides a RESTful API that handles all business logic: user authentication, listing CRUD operations, search/filter queries, and the points system
- Database (MongoDB): Stores user accounts and listing data. The listings collection is populated with the seller's public profile fields on read, so a listing and its seller can be rendered in a single request.

System Flow:

- A user registers or logs in via the authentication system.
- Sellers navigate to the Create Listing page, fill in item details, and submit
    - Listing is stored in the database and appears in the feed.
- Buyers browse the Home Feed or use the search bar to find items, and can apply filters to narrow results.
- Buyers view a listing detail page and contact the seller via Telegram to arrange a meetup.
- After a successful transaction, the buyer will award points to the seller through their profile page.

Description:

- NUSell follows a standard three-tier architecture: a React frontend, a Node.js/Express REST API, and a MongoDB (Atlas) database.
- The frontend never queries the database directly. All reads and writes flow through the Express API over HTTP.
- Every request carries a JSON Web Token (issued at login), which the backend's authentication middleware verifies before any protected route runs.
- Mongoose models (User, Listing) sit between the API and the database, defining the data shape and enforcing validation.

Implementation Philosophy:

- We separated concerns so each tier has a single job: React renders the UI and manages local state, Express handles business logic and authentication, and MongoDB stores the data.
- All requests are funnelled through one api.js (axios) instance, so the JWT is attached in exactly one place and the rest of the frontend stays clean.
- A single middleware checkpoint means authentication is enforced consistently on every protected route, rather than re-implemented route by route.

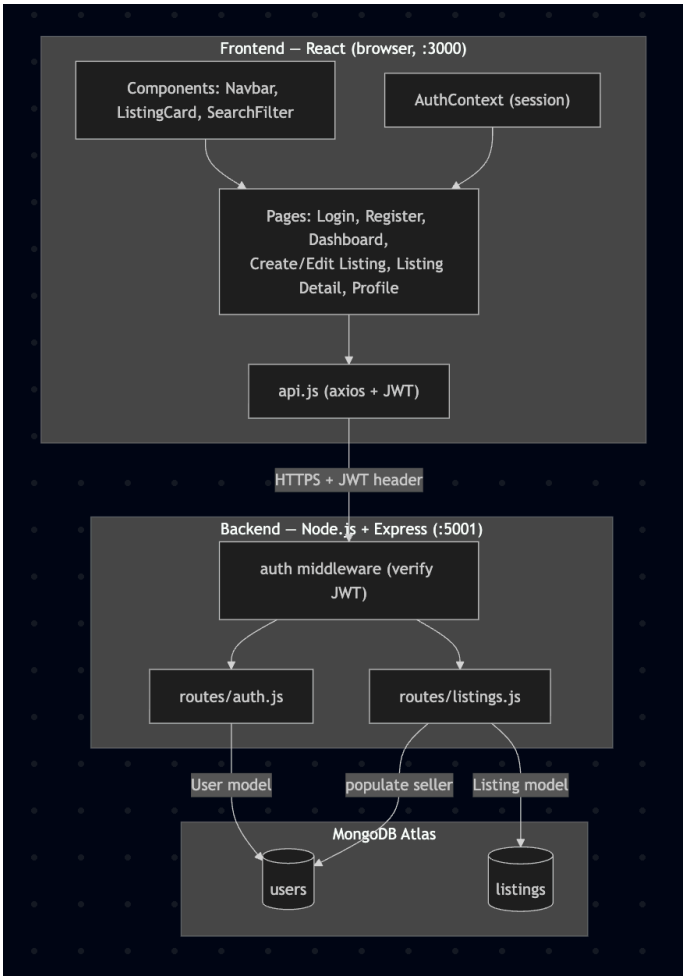


Figure 11: Overall system architecture.

6. Tech Stack

Layer	Choice	Notes
Frontend framework	React	Components: Navbar, ListingCard, SearchFilter. Pages: Login, Register, Dashboard, Create/Edit Listing, Listing Detail, Profile.
HTTP client	axios (api.js)	Single instance; attaches the JWT header to every request.
Backend framework	Node.js + Express (:5001)	auth middleware verifies the JWT before routes/auth.js and routes/listings.js run.
Authentication	JWT + bcrypt	Passwords hashed with bcrypt; stateless JWT verified on every protected route.
Database	MongoDB (Atlas)	users and listings collections, as shown in the system architecture diagram.
Frontend hosting	Vercel	Deployed live at nusell-orbital.vercel.app.

Layer	Choice	Notes
Backend hosting	Render	Deployed on render.
Version control	Git and Github	Repository hosted on Github, with standard branching workflow

7. Software Engineering Practices

Version control & collaboration

Git and Github. Codebase is managed on GitHub, where main contains the deployed build and all active work takes place on feature branches, each tracked through GitHub Issues and merged via Pull Requests (closes #N) after review, keeping main always deployable. Each update is integrated through pull requests, maintaining a reliable branch-and-PR workflow across this milestone.

Modular, layered architecture — clear separation of concerns: React pages/components ↔ api.js service layer ↔ Express routes ↔ Mongoose models, each with a single responsibility.

DRY & reusable components — shared UI (Navbar, ListingCard, SearchFilter) and a single AuthContext + api.js instance as one source of truth for session and requests.

Abstraction — api.js centralises all HTTP + JWT handling so pages never touch tokens; auth middleware centralises access control across every protected route.

Security by design — bcrypt-hashed passwords, JWT-verified middleware, owner-only edit/delete (403 Forbidden), and secrets kept in .env (rotated after a leak) with a committed .env.example template.

Continuous deployment

The frontend is deployed at *https://nus-orbital.vercel.app*, and automatically redeploys from our main branch (via Vercel's Git integration). Backend is hosted on MongoDB Atlas.

Testing

Manual system testing has been carried out against the live deployed application and the MongoDB Atlas database (see System Testing section).

Consistent error handling — try/catch on every route with meaningful HTTP status codes (400 / 403 / 404 / 500) and user-friendly messages

Data validation — Mongoose schema constraints (required, enum, length limits) and server-side checks, backed by client-side form validation.

8. System and Integration Testing

We performed manual system testing by going through each feature in the live deployed application against the MongoDB Atlas database, rather than against local fixtures or mocked data, and simultaneously covered integration testing through ensuring that the frontend, backend and database all work well together. Cases below cover the expected happy paths, permission and ownership checks, and validation edge cases.

#	Feature	Test Case	Expected Result	Outcome
1	Auth (register)	Register a new account with an NUS email, username, Telegram handle, and password	Account created; user redirected to the dashboard	Pass
2	Auth (login)	Log in with correct credentials	Session created, JWT stored, dashboard loads	Pass
3	Auth (invalid login)	Log in with an incorrect password	Clear error message shown; no token issued	Pass
4	Auth (duplicate account)	Register with a username/email already in use	Duplicate error shown; account not created	Pass
5	Password security	Inspect the stored user record in the database	Password stored as a bcrypt hash, never in plaintext	Pass
6	Session persistence	Refresh the page while logged in	Session persists via the stored token, revalidated against GET /api/auth/me	Pass
7	Unauthenticated access	Visit a protected route while logged out	PrivateRoute redirects to the Login page	Pass
8	Create listing	Submit the Create Listing form with valid fields	New listing saved, appears in the gallery, seller redirected to its detail page	Pass
9	Create listing (validation)	Submit the form with a negative price or an empty title	Form blocks submission and shows an inline error	Pass
10	Listing ownership on create	Inspect a newly created listing's seller field	Seller is set from the verified JWT, not from client input	Pass
11	Edit listing (owner)	Owner edits their own listing	Changes saved and reflected on the listing's detail page	Pass
12	Edit listing (non-owner)	Call PUT /api/listings/:id directly as a different user	403 Forbidden returned; listing unchanged	Pass
13	Delete listing (owner)	Owner deletes their own listing	isActive set to false; listing removed from the gallery but preserved in the database	Pass

#	Feature	Test Case	Expected Result	Outcome
14	Delete listing (non-owner)	Call DELETE /api/listings/:id directly as a different user	403 Forbidden returned; listing unchanged	Pass
15	Gallery display	Open the dashboard	Active listings shown as cards with image, name, and price	Pass
16	Search	Search for an item by name	Gallery narrows to matching listings	Pass
17	Filters	Apply category, price range, condition, and location filters together	Gallery narrows correctly across the combined filters	Pass
18	Profile view (own)	Open own profile page	Username, Telegram handle, and active listings shown	Pass
19	Profile view (other user)	Open another user's profile page	Public info and listings shown; email and password excluded	Pass
20	Profile edit	Edit username / Telegram handle via the profile modal	Changes saved; navbar username updates immediately	Pass

9. User Testing

Approach
We ran two rounds of usability testing to validate NUSell's full flow across all five features.

Participants
5 NUS students (our target users), recruited across Year 1-4 and 3 faculties. Each tested the standard features: create a listing (with meetup location, price, condition, contact), search for an item and browse the gallery, view a profile page, list a free item and find it in the giveaway section, and award points to a seller after a transaction.

Key findings

#	Feature	What we observed	Action taken/to take
1	Listing (F1)	3/5 users missed the meetup-location field or left contact blank *(“wasn't sure it was required”)*	Made key fields required + added inline hints
2	Search (F2)	Fast results and generally good feedback. Noticed one minor inconvenience: keypad kept going down while entering price filter	To be addressed during final refinements
3	Giveaway (F4)	Good feedback: intuitive, easy to navigate	Nil
4	Points (F5)	Users unsure how points helped them *(“what’s the use of earning points” / “how does levelling up help?”)*	To be added: information button for points system

Next steps
After Milestone 3: roll out System Usability Scale questionnaire with a larger test cohort ([15+] users), stress-test search with more listings, and refine how points-boosted listings are ranked so visibility feels fair to new sellers.

10. Automated Testing

Alongside the manual system and integration testing described above, we implemented automated tests to verify the core behaviour of NUSell and catch regressions whenever changes are made to the codebase. The automated test suite can be run using ‘npm test’.

Backend Testing
The backend uses Jest as the test framework and Supertest to send HTTP requests directly to the Express application. The tests focus on the authentication and listings APIs, covering both successful requests and important validation, authentication, and permission checks.

The authentication tests cover:

- Registering a new user and receiving a JWT
- Rejecting duplicate accounts
- Logging in with the correct credentials
- Rejecting an incorrect password

The listings tests cover:

- Rejecting listing creation without an authentication token
- Allowing an authenticated user to create a listing
- Returning active listings through the listings API
- Preventing a user from deleting another seller’s listing

These tests also verify the JWT authentication middleware, ensuring protected routes require a valid token and that ownership rules are enforced by the backend rather than only through the frontend.

Frontend Testing
The frontend uses React Testing Library for component-level testing. This allows individual React components and user-facing behaviour to be tested independently, complementing the backend API tests performed with Jest and Supertest.

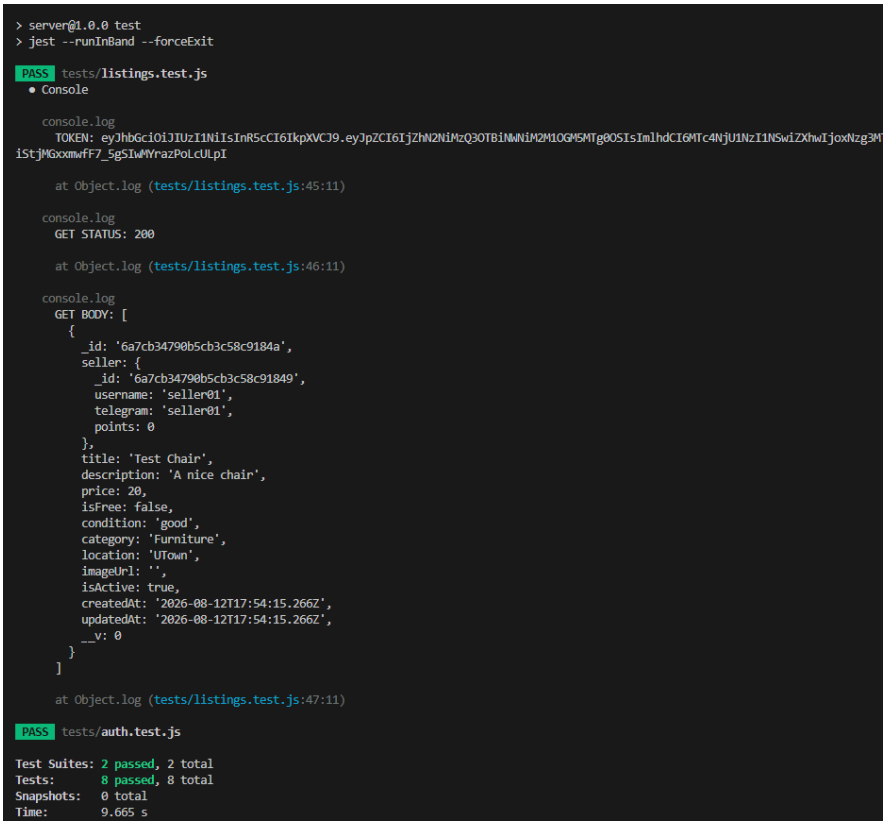


Figure 12: Successful automated test run using Jest and Supertest.

11. Development Plan

Milestone 1 (by 1 June)

Week / Date	Tasks
Week 1–2 (11–25 May)	Set up development environment (Node.js, React, Git repo). Finalise project idea, user stories, and feature list. Create system architecture diagram. Research and learn required technologies.
Week 3 (26 May – 1 Jun)	Build a technical proof of concept: integrated frontend and backend with basic listing creation and feed display. Set up database schemas. Write the Milestone 1 README and project log. Prepare poster and video submission.

Milestone 2 (by 29 June)

Week / Date	Tasks
Week 1–2 (2–15 Jun)	Implement user authentication (sign up, login, logout) with session management. Build full Create/Edit/Delete listing functionality with form validation.

Week / Date	Tasks
Week 3 (16–22 Jun)	Build the browsing feed with pagination. Implement search and filter functionality (category, price range, condition, location).
Week 4 (23–29 Jun)	Build user profile pages showing listings and points. Perform system testing. Fix bugs. Write the Milestone 2 README update and project log.

Milestone 3 (by 27 July)

Week / Date	Tasks
Week 1–2 (30 Jun–13 Jul)	Implement the Giveaway section (listings at $0, dedicated filter page). Build the Points & Reputation system (post-transaction point awarding, displayed on profile).
Week 3 (14–20 Jul)	Implement points-based listing visibility boost in the filtered feed. Add levels and badges to the profile. Polish UI across all pages. Add profile pictures and allow password changes. Remove sellers' own listings from their own gallery view.
Week 4 (21–27 Jul)	Conduct user testing with NUS students. Gather feedback and iterate. Fix remaining bugs. Write the Milestone 3 README update and project log.

Final Submission (by 13 August)

Date	Tasks
28 Jul – 13 Aug	Address feedback from the Milestone 3 peer evaluation and User Testing. Implement automated testing. Final polish and bug fixes. Prepare the A1 poster for physical presentation. Finalise the README and project log.

12. Proof of Concept

The Milestone 2 proof of concept is the deployed, working application rather than a local-only build. Account creation, login, the dashboard and gallery with search/filter, listing creation/editing/deletion, and profile viewing/editing can all be demonstrated end to end (shown under each figure).

Database connection:

The screenshot below shows a live connection to the MongoDB Atlas users collection, with the password field stored as a bcrypt hash rather than plaintext — supporting Test Case 5 in the System Testing section above.

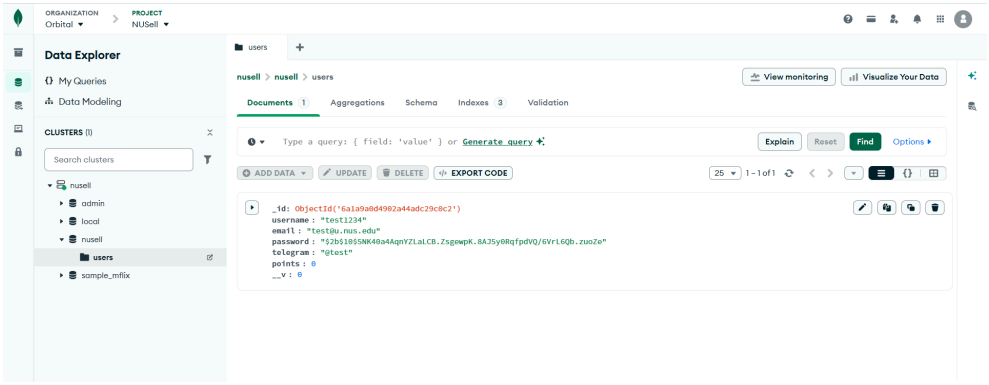


Figure 13: Live connection to the MongoDB Atlas database.

13. Link to our Application

NUSell is hosted live here: ***https://nusell-orbital.vercel.app***